1. Introduction to CPU Scheduling

Modern systems execute multiple programs at the same time. However, a CPU can execute **only one instruction at a time (per core)**.

This creates the need for **decision-making**:

- Which process should run first?
- How long should it run?
- When should it stop?

Formal Definition

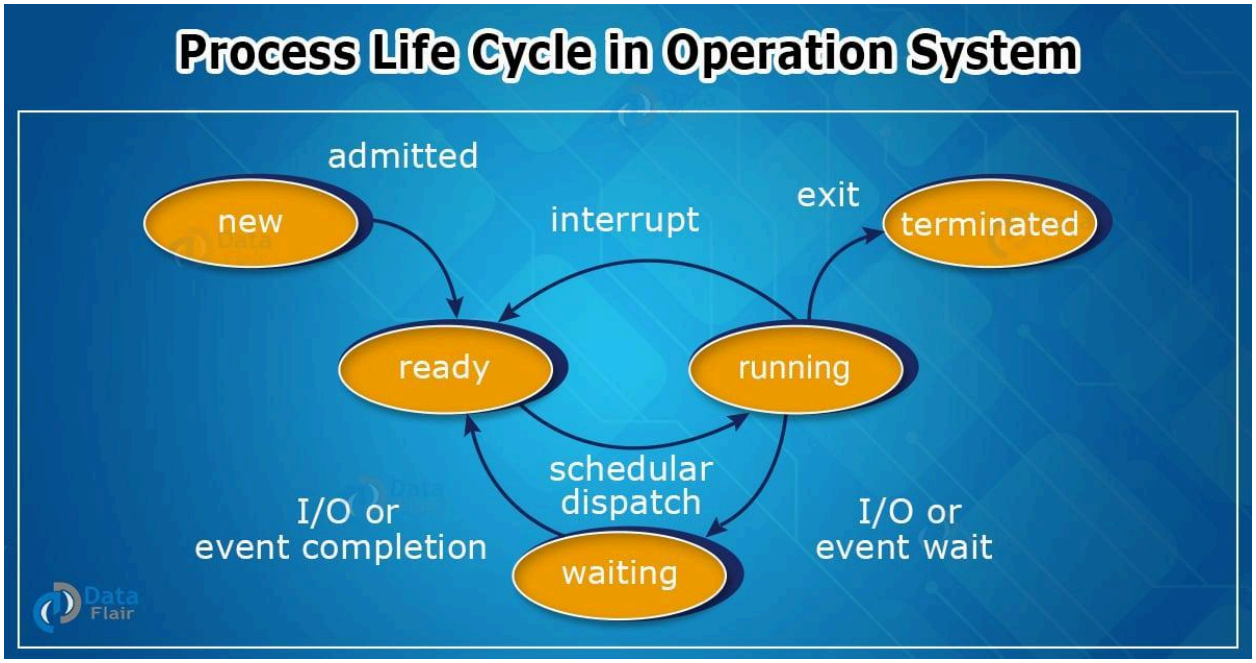
CPU Scheduling is the mechanism used by the Operating System to **select one process from the ready queue and allocate the CPU to it**.

---

2. Process States and Scheduling Context

Understanding scheduling requires understanding **process states**.

Process Lifecycle



Explanation of States

1. **New** – Process is being created
2. **Ready** – Process is waiting for CPU
3. **Running** – Process is executing on CPU
4. **Waiting (Blocked)** – Waiting for I/O
5. **Terminated** – Process finished execution

CPU Scheduling mainly operates on the **Ready Queue**.

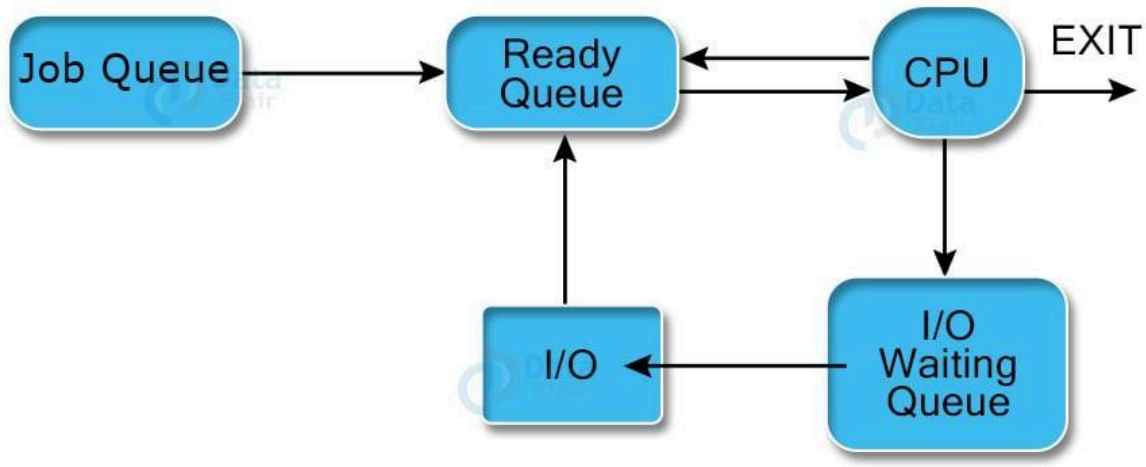
---

3. Ready Queue and CPU Allocation

Definition

The **Ready Queue** is a collection of all processes that are ready to execute but waiting for CPU allocation.

Flow of Execution



Step-by-Step Flow

1. Process enters the system → Job Queue
2. Moves to Ready Queue
3. CPU Scheduler selects process
4. Process executes
5. Either:
    - Completes execution
    - Goes to waiting state

---

4. When Does CPU Scheduling Occur?

CPU scheduling decisions happen in the following **four situations**:

Situation	Explanation
Running → Waiting	Process requests I/O
Running → Ready	Interrupted by OS
Waiting → Ready	I/O completed
Process Termination	Process finishes

Important Concept

- In **Running → Waiting** and **Termination**, CPU must choose next process
- In **other cases**, CPU has **choice**, making scheduling critical

---

5. Types of CPU Scheduling

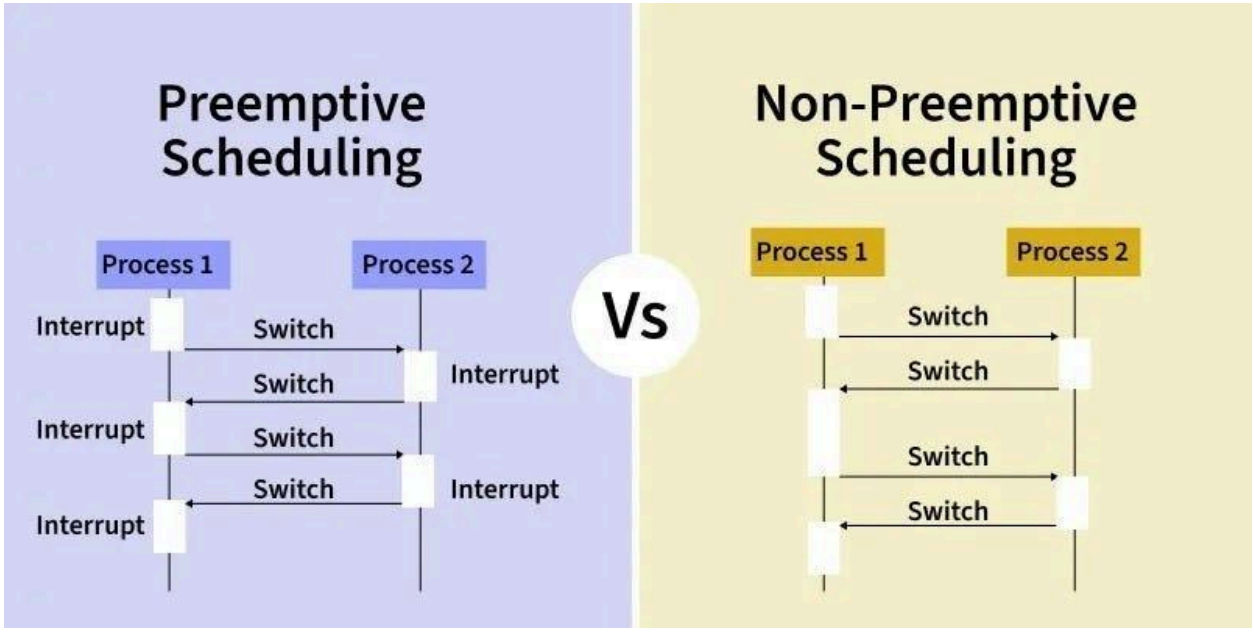
5.1 Non-Preemptive Scheduling

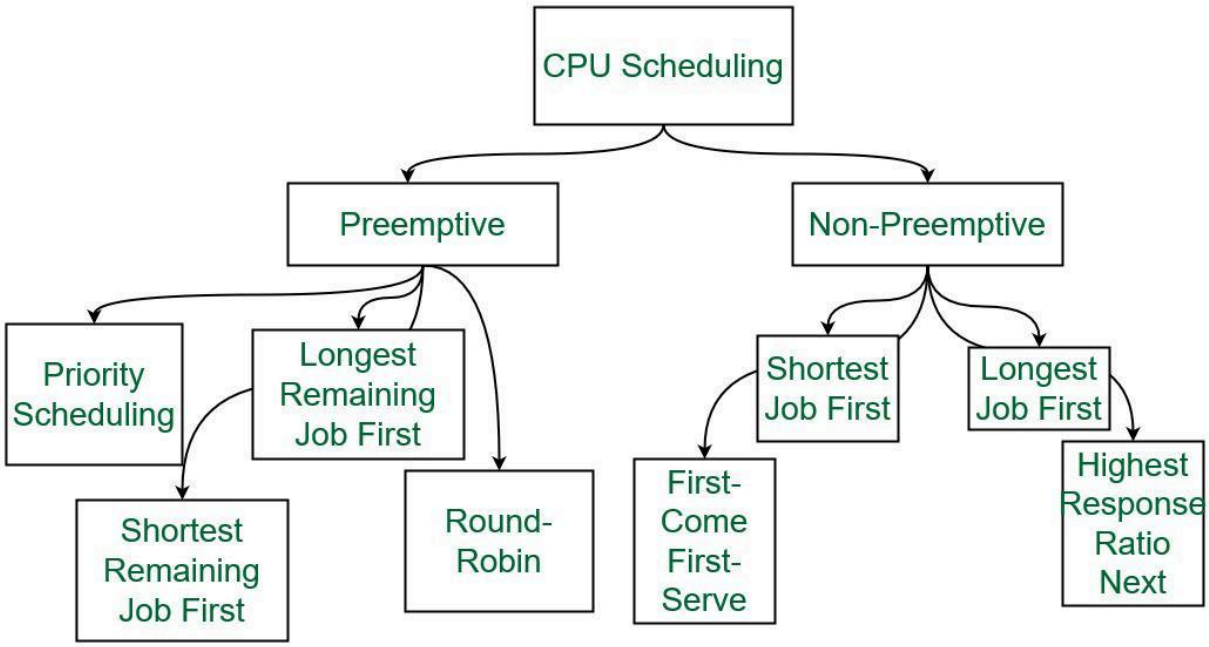
Definition

Once a process starts execution, it **cannot be interrupted** until it:

- Finishes execution, or
- Moves to waiting state

Visual Representation





Key Properties

- Simple implementation
- No interruption

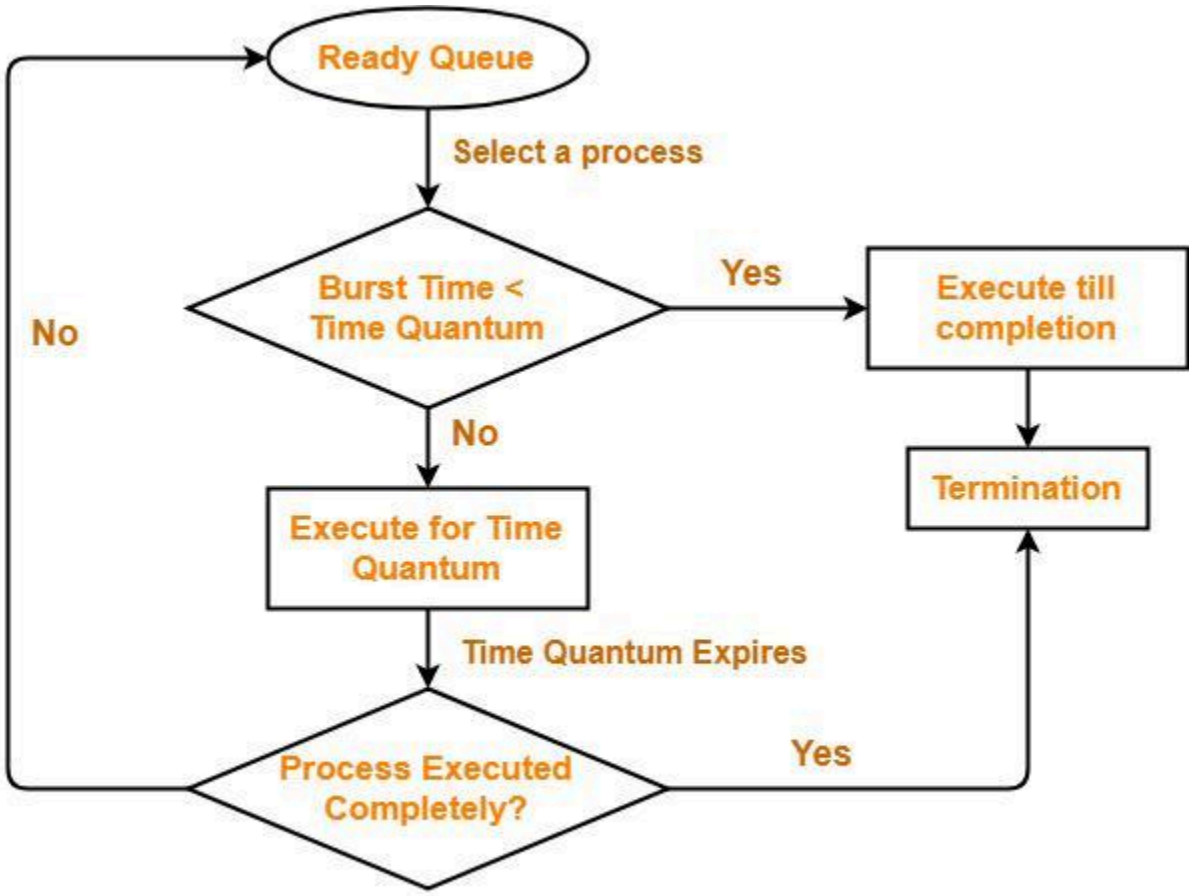
- Can cause long waiting time

5.2 Preemptive Scheduling

Definition

A running process can be **interrupted** and CPU can be assigned to another process.

Visual Representation



Round Robin Scheduling

Key Properties

- Allows interruption

- Improves responsiveness
- Requires context switching

---

6. CPU Scheduling Terminologies

6.1 Arrival Time (AT)

Definition:
Time at which a process enters the ready queue.

Example:
If a process arrives at time 5 ms → AT = 5

6.2 Burst Time (BT)

Definition:
Total CPU time required by a process for execution.

Example:
If execution takes 4 ms → BT = 4

6.3 Completion Time (CT)

Definition:
Time when process finishes execution.

6.4 Turnaround Time (TAT)

Definition:
Total time taken from arrival to completion.

Formula:
TAT = CT − AT

Example:
CT = 10, AT = 2 → TAT = 8

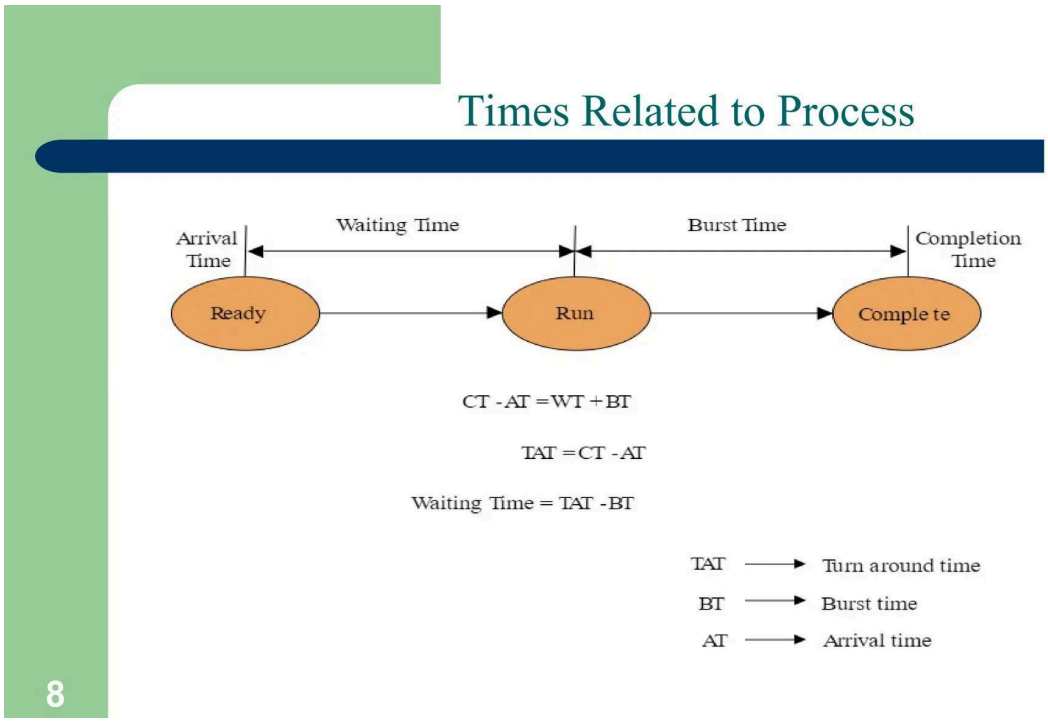
6.5 Waiting Time (WT)

Definition:
Time spent waiting in ready queue.

Formula:
WT = TAT − BT

6.6 Response Time (RT)

Definition:
Time from arrival to **first CPU allocation**.



Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	6	6-0 = 6	6-6 = 0
P2	2	8	17	17-2 = 15	15-8 = 7
P3	4	3	9	9-4 = 5	5-3 = 2

---

7. CPU Scheduling Criteria

7.1 CPU Utilization

Definition:
Percentage of time CPU is actively executing processes.

Goal: Maximize

7.2 Throughput

Definition:
Number of processes completed per unit time.

7.3 Turnaround Time

Definition:
Total execution time of process

Goal: Minimize

7.4 Waiting Time

Definition:
Time spent waiting before execution

Goal: Minimize

7.5 Response Time

Definition:
Time taken to get first response

Important for: Interactive systems

---

8. CPU Scheduling Algorithms

8.1 First-Come, First-Served (FCFS)

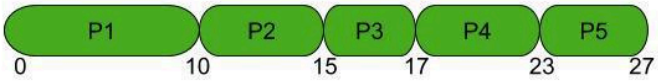
Definition

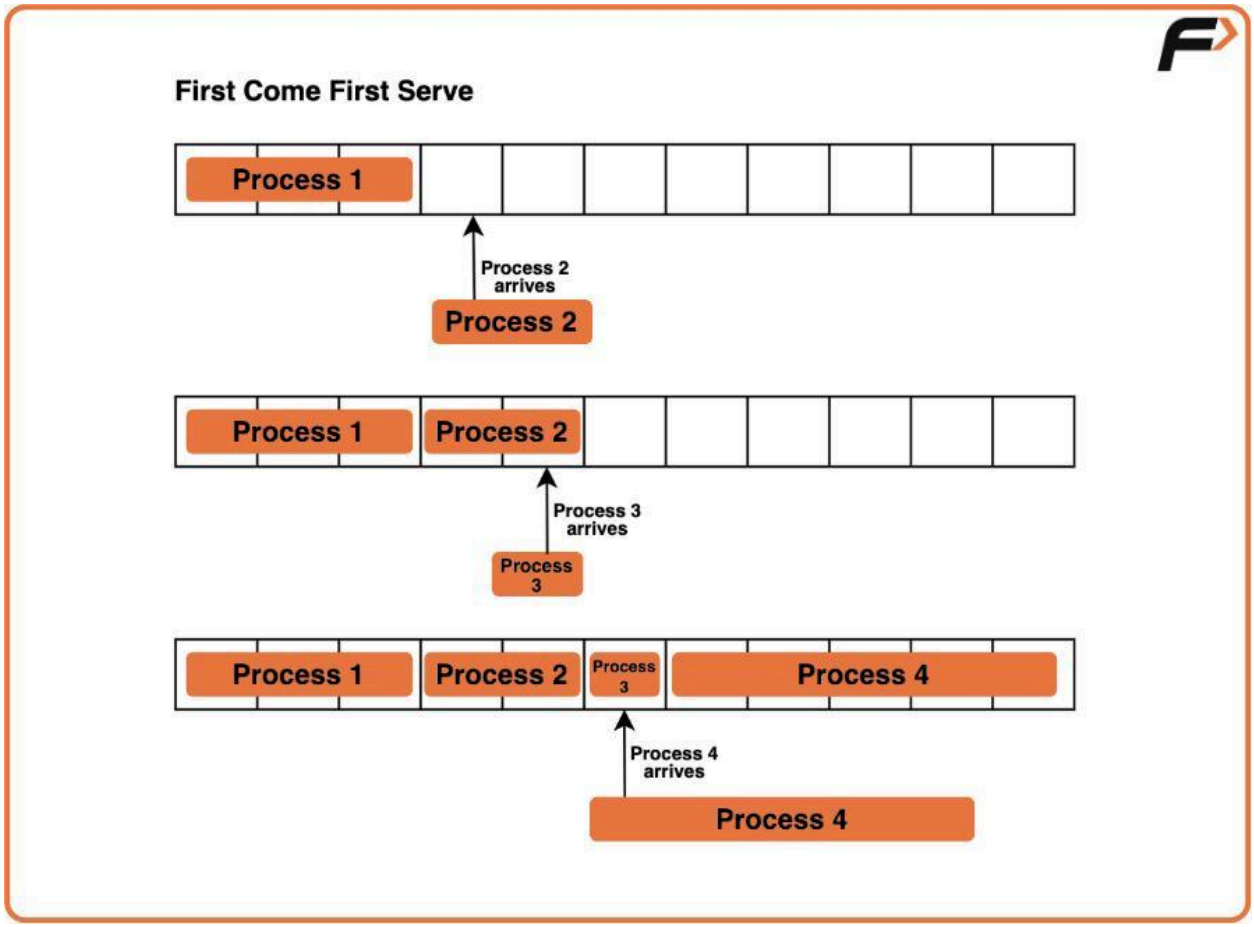
Processes execute in order of arrival.

Type None - Preemptive

PROCESS	ARRIVAL TIME	BURST TIME
P1	0	10
P2	3	5
P3	5	2
P4	6	6
P5	8	4

GANTT CHART





Key Points

- Simple
- No starvation
- Suffers from **Convoy Effect** (The Convoy Effect is a situation in CPU scheduling where a long process occupies the CPU first, forcing many short processes to wait behind it, leading to poor overall performance.)

Example

| P1 (5) → P2 (3) → P3 (2) |

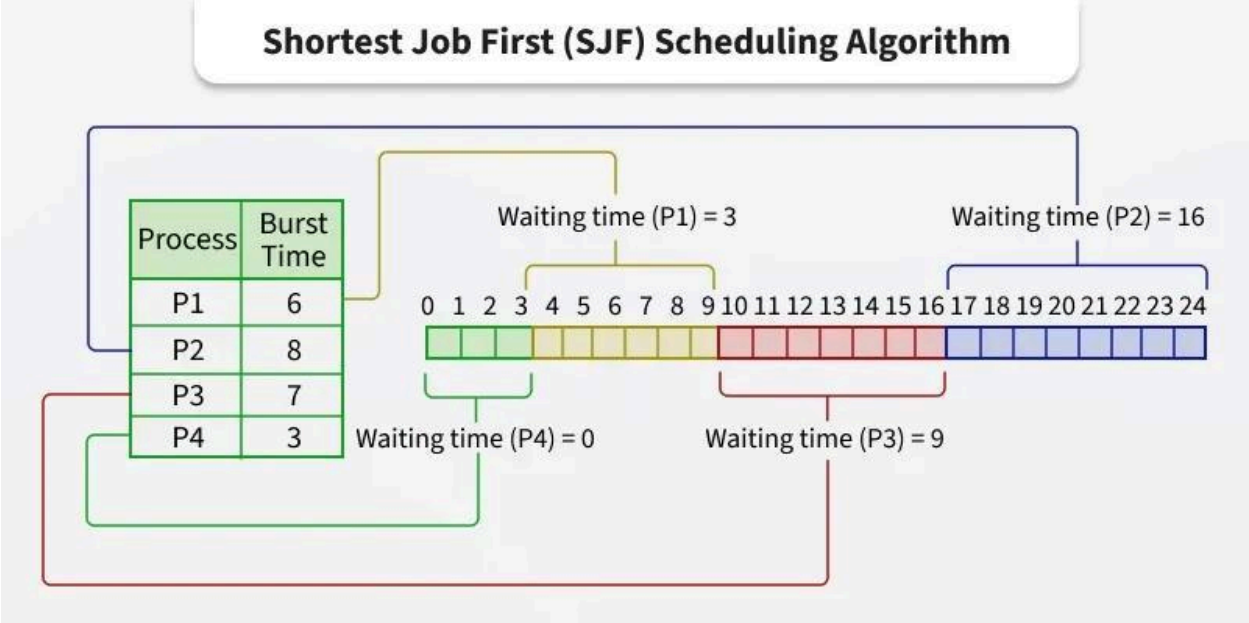
8.2 Shortest Job First (SJF)

Definition

Process with **smallest burst time** executes first.

Types

- Non-preemptive
- Preemptive (Shortest Remaining Time First)



Advantages and Disadvantages

Advantages

- Shorter jobs first – less waiting time than FCFS and SJF
- Higher CPU utlilization and throughput than FCFS and SJF
- No convoy effect

Disadvantages

- Can lead to starvation
- Tough to implement (predicting next CPU burst time)

Key Points

- Minimum average waiting time
- Requires burst time prediction
- Can cause starvation

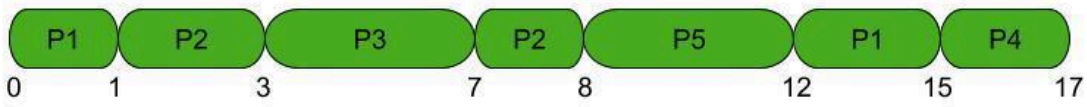
8.3 Priority Scheduling

Definition

Each process has a priority. Higher priority executes first.

PROCESS	ARRIVAL TIME	BURST TIME		PRIORITY
		TOTAL	REMAINING	
P1	0	4	4	4
P2	1	3	3	3
P3	3	4	4	1
P4	6	2	2	5
P5	8	4	4	2

GANTT CHART:



Key Points

- Important tasks handled first
- Starvation possible

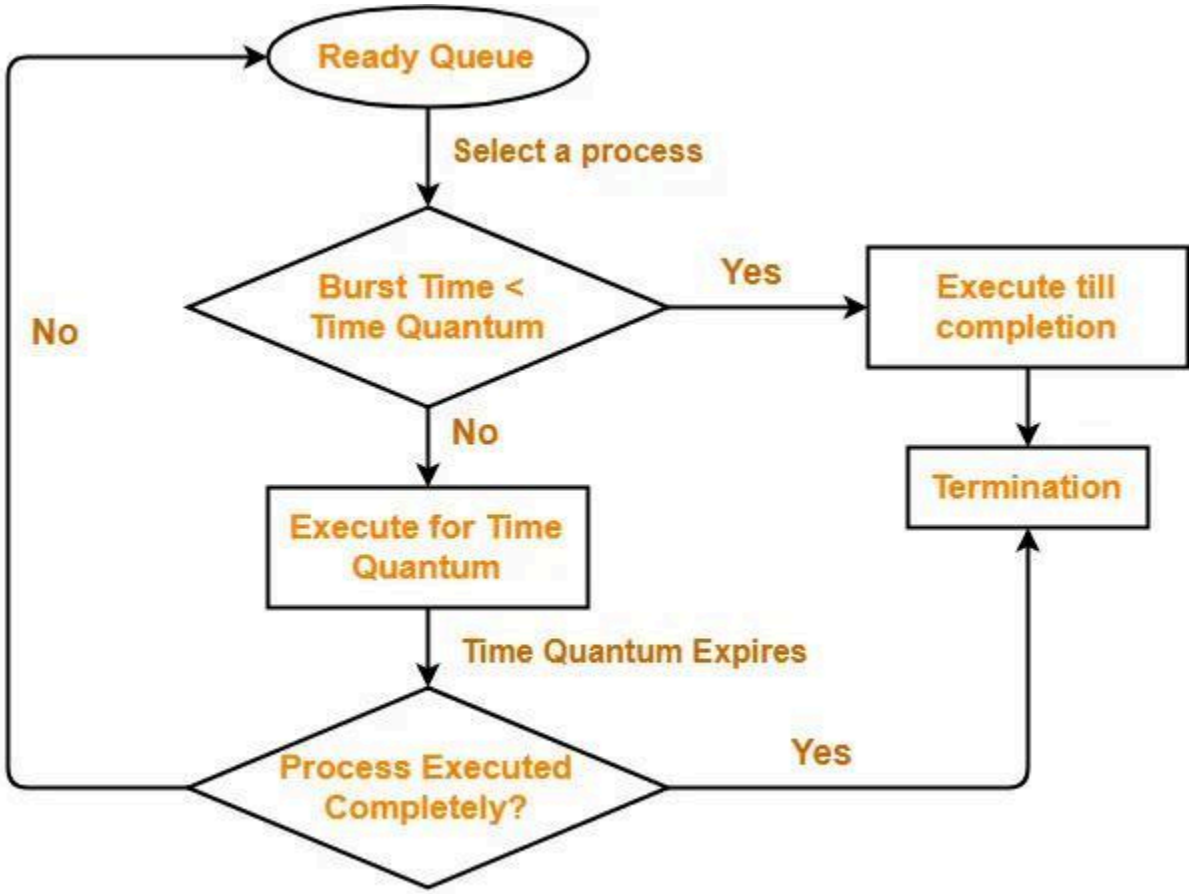
Solution

Aging: Increase priority over time

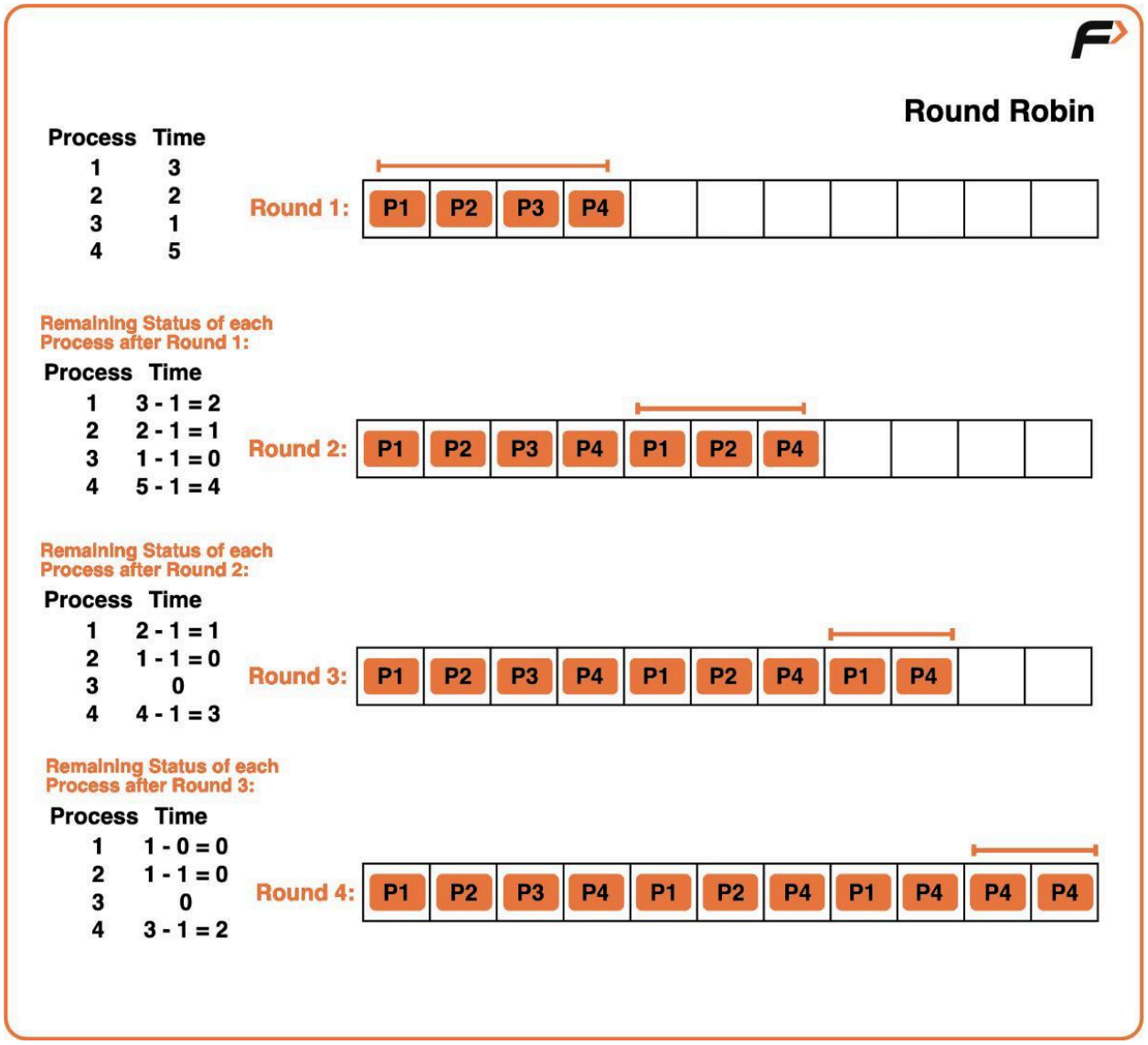
8.4 Round Robin Scheduling

Definition

Each process gets fixed time slice (time quantum).



Round Robin Scheduling



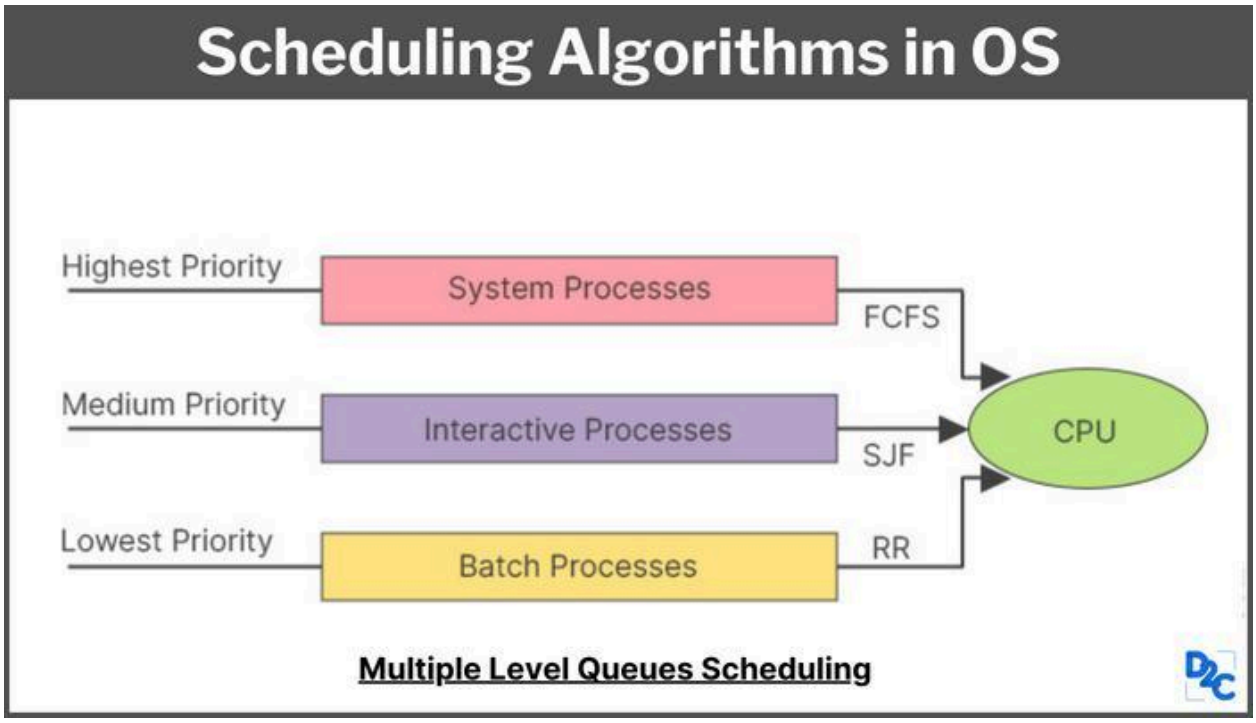
Key Points

- Fair scheduling
- Used in time-sharing systems
- Trade-off:
    - Small quantum → overhead
    - Large quantum → FCFS behavior

8.5 Multilevel Queue Scheduling

Definition

Processes divided into different queues based on type.



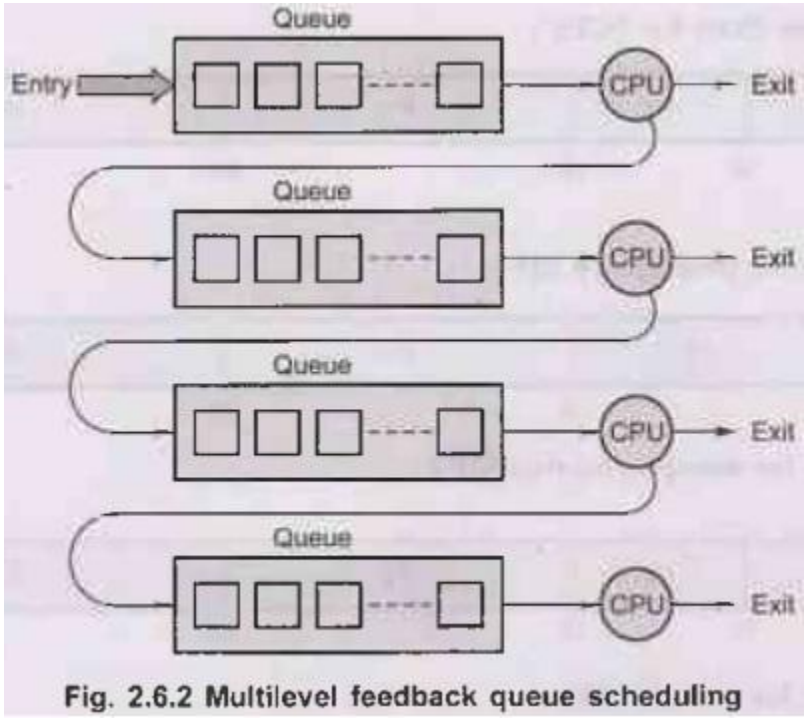


Fig. 2.6.2 Multilevel feedback queue scheduling

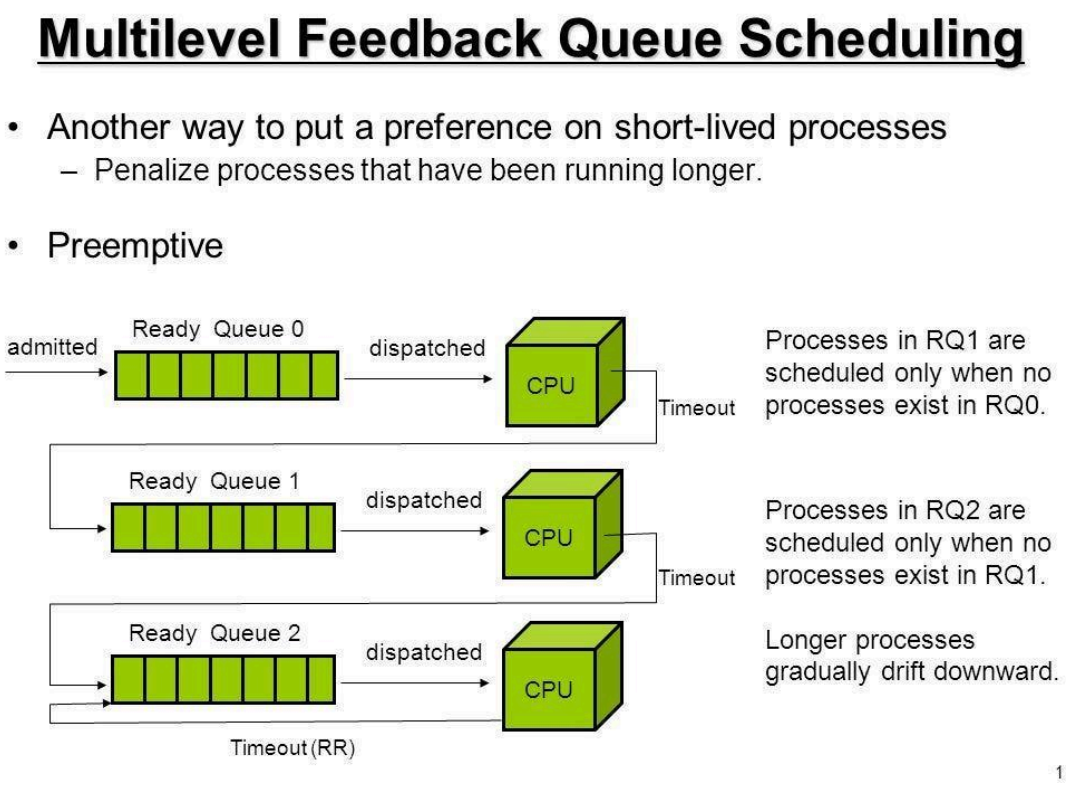
Key Points

- Each queue has its own algorithm
- Fixed priority between queues
- No movement between queues

8.6 Multilevel Feedback Queue (MLFQ)

Definition

Processes can move between queues based on behavior.



Key Points

- Dynamic priority adjustment
- CPU-heavy → lower priority
- Interactive → higher priority
- Prevents starvation

---

9. Solving Numerical Problems (Step-by-Step Method)

Always follow this order:

1. Draw Gantt Chart
2. Calculate Completion Time
3. Calculate Turnaround Time
4. Calculate Waiting Time
5. Compute averages

---

10. Final Summary

- CPU scheduling ensures efficient CPU usage
- Multiple algorithms exist with different trade-offs
- No single algorithm is optimal for all cases
- Understanding **when and why to use each algorithm** is critical